

Top 10 web hacking techniques of 2024



James Kettle
Director of Research
[@albinowax](#)



Published: Tuesday, 4 February 2025 at 15:01 UTC

Updated: Tuesday, 4 February 2025 at 15:20 UTC



Welcome to the Top 10 Web Hacking Techniques of 2024, the [18th edition](#) of our annual community-powered effort to identify the most innovative must-read web security research published in the last year.

This post is the culmination of a three-step collaboration with the security community. Over the last month:

- The community [submitted nominations](#) for the top research from 2024
- The community voted on those nominations to build a shortlist of the top 15
- An expert panel voted on the shortlist to select and order the 10 finalists

This year, the community nominated a staggering 121 pieces of research - nearly double what we saw last time. To make the number of options in the community vote manageable, I filtered out entries consisting of articles published outside 2024 or outside the scope of [web application security](#), and writeups that, while valuable, were not innovative. Even after this filter, there were 103 entries remaining!

After the community vote, we were honoured to see the top fifteen included three techniques by PortSwigger Research. To avoid risking a repeat of last year, I excluded these from the panel vote. Of course, we are still very proud of them, and you can read them here:

- [Gotta cache 'em all: bending the rules of web cache exploitation](#)
- [Splitting the email atom: exploiting parsers to bypass access controls](#)
- [Listen to the whispers: web timing attacks that actually work](#)

The fifteen finalists from the community vote were then analyzed and voted on by an expert panel consisting of [Nicolas Grégoire](#), [Soroush Dalili](#), [STÖK](#), [Fabian \(LiveOverflow\)](#), and [myself](#).

This year, a single theme dominated the top five - you might be able to guess what it was.

Let's begin the countdown!

10. Hijacking OAuth flows via Cookie Tossing

In tenth place, [Hijacking OAuth flows via Cookie Tossing](#) by [Elliot Ward](#) introduces a novel application of the widely under-estimated Cookie Tossing technique. This research was directly inspired by [an earlier post](#) by [Thomas Houhou](#)

Both articles are essential reading, especially if you ever find yourself stuck with a self-XSS, or XSS in an inconsequential subdomain. Cookies predate the [Same-Origin Policy](#) that governs JavaScript, and this research shows that in spite of decades of security-bodges from [HttpOnly](#) to [SameSite](#), they're still a hazard. Maybe it would be safer just to use [localStorage](#) for session tokens instead.

9. ChatGPT Account Takeover - Wildcard Web Cache Deception

[Web Cache Deception](#) originally debuted at #2 in the [top web hacking techniques of 2017](#), and has recently seen rapid development.

In [ChatGPT Account Takeover - Wildcard Web Cache Deception](#), [Harel](#) introduces a twist on the technique, exploiting inconsistent decoding to perform [path traversal](#) and escape a cache rule's intended scope. We built a Web Security Academy lab based on this technique, so you can [try it out for yourself](#).

We highly recommend reading all the author's writeups - they were a fundamental inspiration for our own web cache deception research.

8. OAuth Non-Happy Path to ATO

In position 8, [OAuth Non-Happy Path to ATO](#) by [Oxrz](#) articulates the thought process behind a beautiful and innovative attack chain. STÖK perfectly captured why this research stands out:

“ I just love how something as seemingly benign as an app honoring a manipulated "Referer:" header can turn into a full-blown account takeover via OAuth. This chain perfectly demonstrates how inspiration from prior research (in this case, [Frans Rosén's](#) almost legendary [Dirty Dancing](#) write-up) combined with a deep dive into the OAuth documentation can lead to some seriously creative attack chains. I had completely forgotten about this attack flow, but there's no way I'm not automating checks for referer-based redirects whenever I'm poking at stuff from now on! ”

7. CVE-2024-4367 - Arbitrary JavaScript execution in PDF.js

In seventh place, we've got... a CVE! [CVE-2024-4367 - Arbitrary JavaScript execution in PDF.js](#) to be precise. It's rare that a single, patched vulnerability makes its way into the top ten, but this finding by [Thomas Rinsma](#) is exceptional. PDF.js is widely embedded as a library, making the second-order impact both huge and difficult to predict. This research is a quality analysis of some severely overlooked attack surface, and undermines assumptions about where an attacker might get a foothold.

If you enjoy PDF shenanigans like this, we highly recommend reviewing publications by [Alex Inführ & Ange Albertini](#).

6. DoubleClickjacking: A New Era of UI Redressing

[DoubleClickjacking: A New Era of UI Redressing](#) introduces a variation on Clickjacking that bypasses pretty much every known mitigation. This entry proved controversial with the panel because it seems simple and deceptively obvious in retrospect, but still came in highly placed due to raw, undeniable value.

While glimmers of this attack concept have existed for years, [Paulos Yibelo](#) delivers it with a perfect execution that proves it's unequivocally the right time for this attack. Framing restrictions and SameSite cookies have largely killed Clickjacking, and browser performance has achieved a level that makes the sleight of hand pretty much invisible. Love it, hate it, or simply hate the fact that you didn't discover it first, this is not a technique to ignore!

5. Exploring the DOMPurify library: Bypasses and Fixes

HTML sanitisation has been an XSS battleground for decades, and the DOMPurify library by Cure53 has emerged as pretty much the only defensive solution that actually works.

[Exploring the DOMPurify library: Bypasses and Fixes](#) dives deep into browser HTML-parsing internals, discovering and applying novel mutation XSS (mXSS) primitives. Described by LiveOverflow as "An absolute joy to read" and "Probably the most comprehensive article for understanding mXSS and how this affects sanitizers such as DOMPurify", this is a must-read for anyone into JavaScript and XSS, and will serve as a manual for anyone looking to develop a HTML sanitisation bypass for years to come.

Awesome work by [Mizu](#).

4. WorstFit: Unveiling Hidden Transformers in Windows ANSI

Everyone 'knows' that charset conversion is an absolute minefield, and yet somehow it's rarely seen in real exploits. In [WorstFit: Unveiling Hidden Transformers in Windows ANSI](#), [Orange Tsai](#) and [splitline](#) prove the true power of this attack class, racking up numerous CVEs and triggering a vendor blame-game in the process. It's always a sign of great research when something that seems like it should be fundamental platform knowledge pops up and takes everyone by surprise.

We expect to see more discoveries in this area, and after catching this talk live at Black Hat Europe I pushed automatic detection of WorstFit-style transformations into ActiveScan++ to help out. STÖK spotted the [WorstFit mapping explorer](#) is an absolute gem for generating fuzzing wordlists, too.

3. Unveiling TE.0 HTTP Request Smuggling

The community's understanding of request smuggling is still rapidly evolving, and [Unveiling TE.0 HTTP Request Smuggling: Discovering a Critical Vulnerability in Thousands of Google Cloud Websites](#) is a major, must-read contribution by [Paolo Arnolfo](#), [Guillermo Gregorio](#), and [@_medusa_1_](#)

This research is personally significant for me as it taught me an important lesson. Back when I first encountered CL.0 request smuggling, I hypothesized that TE.0 could exist but that it would never be exploitable, as it would require the back-end server to accept a HTTP request starting with a number and a newline. I was very, very, wrong. Once you've mastered the fundamentals, if you want to push the boundaries, relying on prediction and analysis can hold you back. If you don't ask the question because you think you know the answer, you stay ignorant.

If you're wondering how the attack actually works, my best guess is that the front-end was rewriting the body as non-chunked, but forgetting to set the Content-Length header due to the OPTIONS method. This is an insane finding which opens the door to a whole lot of possibilities. Watch this space.

2. SQL Injection Isn't Dead: Smuggling Queries at the Protocol Level

Sometimes you can tell research is going to be amazing just from the subtitle. LiveOverflow has a great analysis:

“Great research progress often happens at the intersection of fields. In [Paul Gerste's SQL Injection Isn't Dead Smuggling Queries at the Protocol Level](#) we can see binary memory corruption ideas being applied to the world of web hacking. We have an integer overflow that corrupts a size, and basically a heap-spray technique to hit a fake Query more reliably... beautiful.”

It's a testament to how strong the competition was this year that this didn't grab first place.

1. Confusion Attacks: Exploiting Hidden Semantic Ambiguity in Apache HTTP Server

[Orange Tsai](#) has claimed the #1 position for the third time with [Confusion Attacks: Exploiting Hidden Semantic Ambiguity in Apache HTTP Server](#). This inspiring, deep and impactful research publication left the entire panel in awe. Here's what they had to say:

“Once again, some fantastic research by Orange! It's crazy nobody considered approaching Apache in this way before! - Nicolas

“I'm certain we're just scratching the surface of what's possible by building on this research. Can't wait to dig deeper, hunt for fingerprints and indicators of confusions and when the time is right, go all brrrrrr! - STÖK

“Orange Tsai treats Apache httpd like a web CTF challenge! It's incredible how deep and impactful Orange's research (always) is. Given the popularity of httpd, this research will serve as a reference for security practitioners for a long time. - LiveOverflow

“Orange is confusing all the apps! - Soroush

This is incredible, must-read research and absolutely deserves top place. Congratulations Orange!

Conclusion

The security community published a record-breaking amount of high-quality research in 2024, leading to intense competition for both the community and panel votes. This wasn't just a matter of quantity - this was the highest quality crop of research I've seen since picking up the top ten project in 2018, and if the trend continues next year it's going to cause carnage. With 103 nominations and only ten spots, many great writeups didn't make the cut, so be sure to check out the [full nomination list](#) and let us know what your #1 was. Also, if you spotted some exceptional research from 2024 that never got nominated, chuck me an email and I'll add it to the list.

Part of what lands an entry in the top 10 is its expected longevity, so it's well worth getting caught up with [the top ten archive](#) too. If you're interested in getting a preview of what might win from 2025, you can [subscribe to our RSS](#), join [r/websecurityresearch](#), hop on [our Discord](#), or follow us on social. If you're interested in doing this kind of research yourself, I've shared a few lessons I've learned over the years in [Hunting Evasive Vulnerabilities](#), [How to choose a security research topic](#), and [So you want to be a web security researcher?](#)

Massive thanks to the panel for contributing their time and expertise to curating the final result, and thanks also to everyone who took part! Without your nominations, votes, and most-importantly research, this wouldn't be possible.

Till next time!

[Top 10 Hacking Techniques](#)

[← Back to all articles](#)

Related Research



Burp Suite

Web vulnerability scanner
Burp Suite Editions
Release Notes

Vulnerabilities

Cross-site scripting (XSS)
SQL injection
Cross-site request forgery
XML external entity injection
Directory traversal
Server-side request forgery

Customers

Organizations
Testers
Developers

Company

About
Careers
Contact
Legal
Privacy Notice
Modern Slavery Statement

Insights

Web Security Academy
Blog
Research
Engineering

 **PortSwigger**

 [Follow us](#)

© 2026 PortSwigger Ltd.

